Introduction to Programming Language with AI

JavaScript + Node.js

Name: Rabeya **Bosri** **Course:** SQA: Manual & Automated Testing

Q1. JavaScript Coding Practice

Program 1: Check Whether a Number Is Even or Odd

Problem Statement

Write a JavaScript program that takes a number and determines whether it is even or odd.

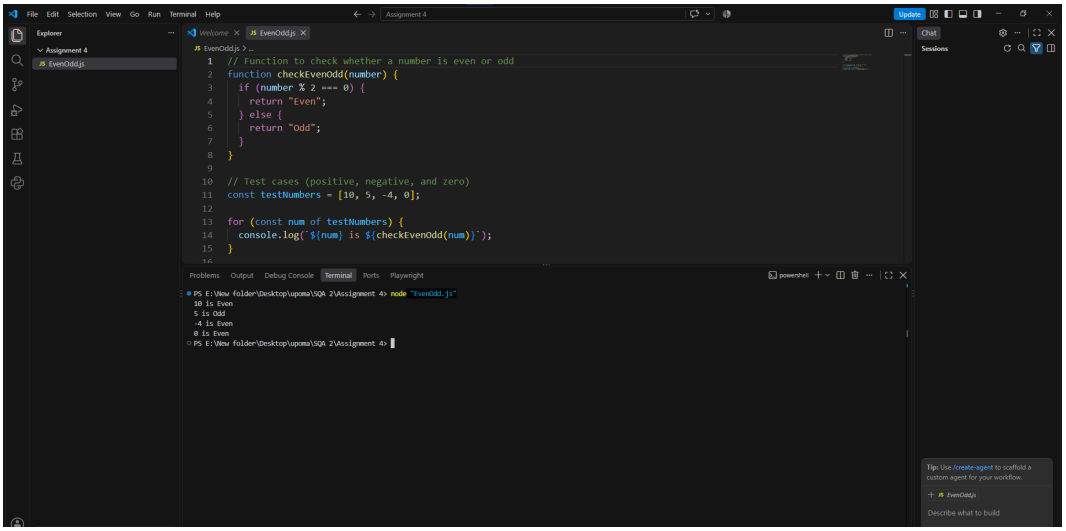
Source Code (evenOdd.js)

```
function checkEvenOdd(number) {
  // A number divisible by 2 with no remainder is even
  if (number % 2 === 0) {
    return "Even";
  } else {
    return "Odd";
  }
}

// Test cases (positive, negative, and zero)
const testNumbers = [10, 7, -4, 0];

for (const num of testNumbers) {
  console.log(`${num} is ${checkEvenOdd(num)}`);
}
```

Output



Explanation

- The **modulus operator (`%`)** returns the remainder of a division. `10 % 2` is `0`, so 10 is even. `7 % 2` is 1, so 7 is odd.
- The function checks `number % 2 === 0`. `=== 0` is used rather than `=== 1` on purpose: in JavaScript, negative odd numbers give `-1` (for example, `-7 % 2` is `-1`), so checking for `1` would wrongly classify them as even.
- Zero is mathematically even, and the program handles it correctly.
- Testing with a positive number, a negative number, and zero is how a QA engineer covers boundary and edge cases.

Program 2: Find the Largest of Three Numbers

Problem Statement

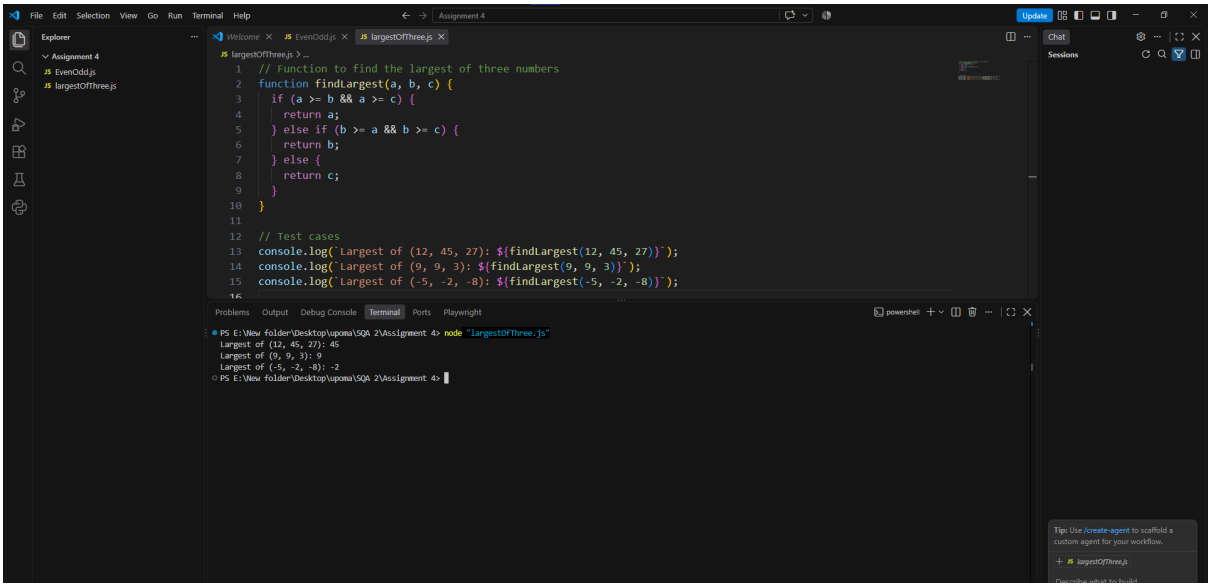
Write a JavaScript program that accepts three numbers and finds the largest among them.

Source Code (largestOfThree.js)

```
function findLargest(a, b, c) {
  if (a >= b && a >= c) {
    return a;
  } else if (b >= a && b >= c) {
    return b;
  } else {
    return c;
  }
}

// Test cases
console.log(`Largest of (12, 45, 27): ${findLargest(12, 45, 27)}`);
console.log(`Largest of (9, 9, 3): ${findLargest(9, 9, 3)}`);
console.log(`Largest of (-5, -2, -8): ${findLargest(-5, -2, -8)}`);
```

Output



Explanation

- The program uses **if / else if / else** and compares the numbers with the logical AND operator (&&).
- If a is greater than or equal to both b and c, then a is the largest. Otherwise, the same check is made for b. If neither passes, c must be the largest.
- Using >= instead of > makes the program handle **equal values** correctly, as in the test case `(9, 9, 3)`.
- The third test case confirms it works with **negative numbers**.
- A shorter alternative is `Math.max(a, b, c)`. The manual approach is used here to demonstrate conditional logic.

Program 3: Reverse a String

Problem Statement

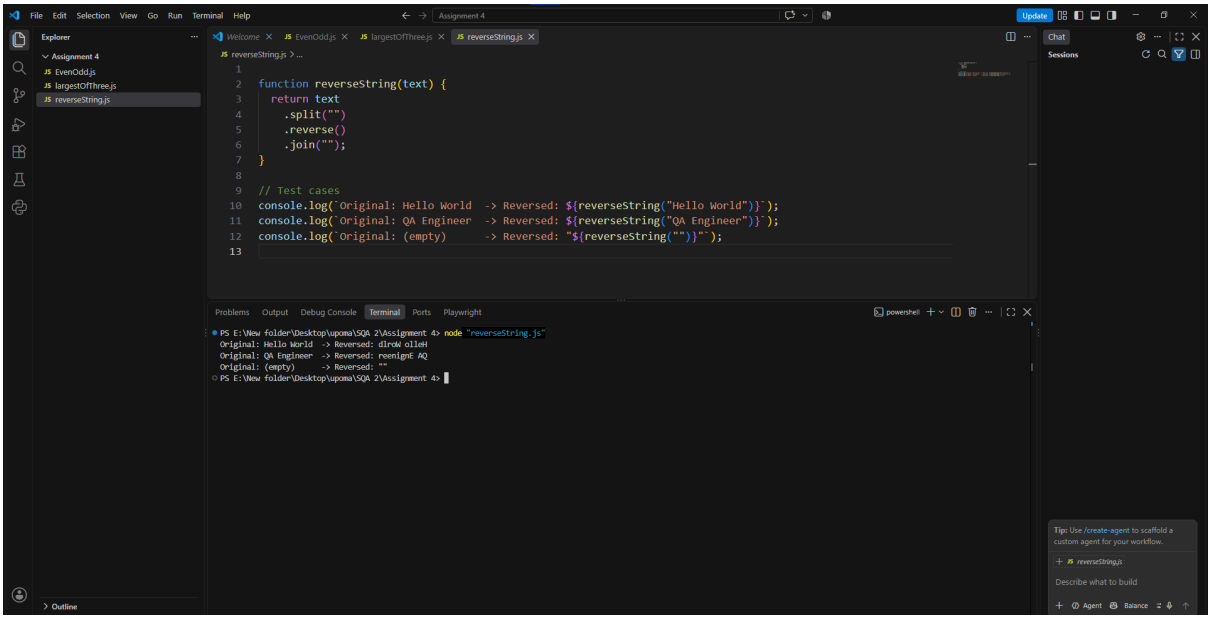
Write a JavaScript program that reverses a given string.

Source Code (reverseString.js)

```
function reverseString(text) {
  return text
    .split("")
    .reverse()
    .join("");
}

// Test cases
console.log(`Original: Hello World  -> Reversed: ${reverseString("Hello World")}`);
console.log(`Original: QA Engineer  -> Reversed: ${reverseString("QA Engineer")}`);
console.log(`Original: (empty)      -> Reversed: "${reverseString("")}"`);
```

Output



Explanation

- JavaScript strings are immutable and have no built-in `reverse()` method, so the string is converted to an array, reversed, and converted back.
- `split("")` turns `"Hello"` into `["H", "e", "l", "l", "o"]`.
- `reverse()` reverses the array in place: `["o", "l", "l", "e", "H"]`.
- `join("")` merges the array into `"olleH"`.
- The empty-string test shows the function does not crash on empty input, which is a standard negative test in QA.

Program 4: Count Vowels in a String

Problem Statement

Write a JavaScript program that counts the number of vowels (a, e, i, o, u) in a given string.

Source Code (countVowels.js)

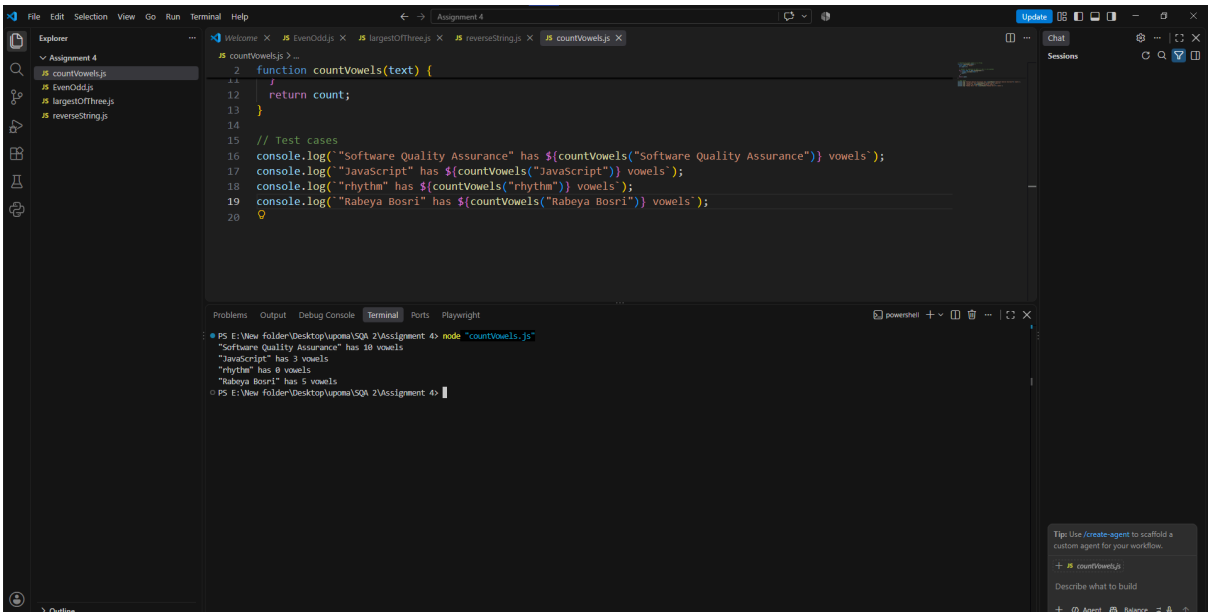
```
function countVowels(text) {
  const vowels = "aeiou";
  let count = 0;

  // Convert to lowercase so both 'A' and 'a' are counted
  for (const char of text.toLowerCase()) {
    if (vowels.includes(char)) {
      count++;
    }
  }
  return count;
}

// Test cases
```

```
console.log(`"Software Quality Assurance" has ${countVowels("Software Quality
Assurance")} vowels`);
console.log(`"JavaScript" has ${countVowels("JavaScript")} vowels`);
console.log(`"rhythm" has ${countVowels("rhythm")} vowels`);
console.log(`"Rabeya Bosri" has ${countVowels("Rabeya Bosri")} vowels`);
```

Output



Explanation

- The string `vowels = "aeiou"` holds the characters to look for.
- `text.toLowerCase()` makes the check **case-insensitive**, so uppercase vowels are counted too.
- The `for...of` loop visits each character. If `vowels.includes(char)` is true, the counter increases.
- Manual verification of the first test: Software (o, a, e = 3) + Quality (u, a, i = 3) + Assurance (A, u, a, e = 4) = **10**.
- The `"rhythm"` test confirms the program returns `0` when no vowels exist, and the last test confirms uppercase and lowercase are treated alike.

Program 5: Remove Duplicate Values from an Array

Problem Statement

Write a JavaScript program that removes duplicate values from an array and returns only the unique values.

Source Code (removeDuplicates.js)

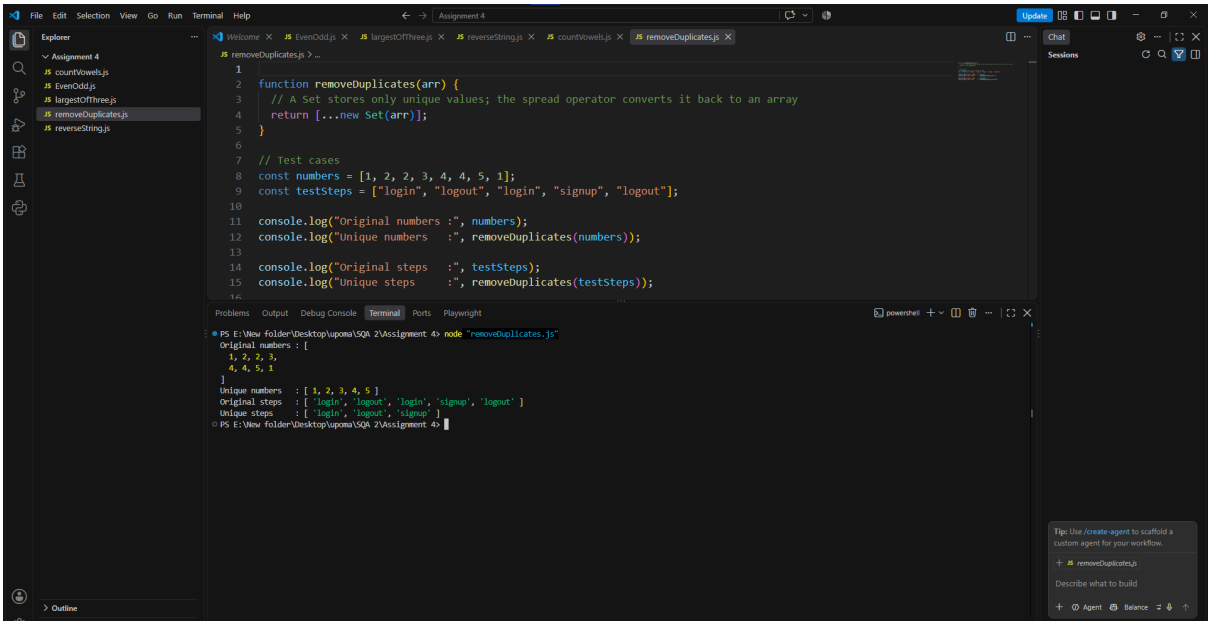
```
function removeDuplicates(arr) {
  // A Set stores only unique values; the spread operator converts it back to an
array
  return [...new Set(arr)];
}
```

```
// Test cases
const numbers = [1, 2, 2, 3, 4, 4, 5, 1];
const testSteps = ["login", "logout", "login", "signup", "logout"];

console.log("Original numbers :", numbers);
console.log("Unique numbers   :", removeDuplicates(numbers));

console.log("Original steps   :", testSteps);
console.log("Unique steps     :", removeDuplicates(testSteps));
```

Output



Explanation

- A **`Set`** is a built-in JavaScript collection that automatically ignores duplicate values.
- `new Set(arr)` creates a Set from the array, discarding repeats.
- The **spread operator** `[...set]` converts the Set back into a normal array.
- The **order of first occurrence is preserved**, so `[1, 2, 2, 3, ...]` becomes `[1, 2, 3, ...]`.
- Numbers and strings both work, and the second test shows a realistic QA use case: cleaning a list of repeated test steps.

Q2. Introduction to Node.js

2.1 What Is Node.js?

Node.js is an open-source, cross-platform **JavaScript runtime environment** that lets you run JavaScript code **outside the web browser**, such as on a laptop, a server, or a CI/CD machine. It was created by Ryan Dahl in 2009.

Two clarifications matter here:

- Node.js is **not a programming language**. The language is JavaScript, and Node.js is the environment that executes it.
- Node.js is **not a framework**. It is a runtime, and frameworks such as Express.js are built on top of it.

Before Node.js, JavaScript could only run inside browsers. Node.js made it possible to use JavaScript for backend development, command-line tools, APIs, and test automation.

Real-world context: In a company, a QA engineer runs a command like `node test.js` or `npx playwright test` on their machine or on a Jenkins/GitHub Actions server. Node.js is the engine executing those tests.

2.2 How Does Node.js Work?

Node.js is built from several cooperating components.

Component 1: V8 JavaScript Engine

Node.js uses **V8**, the same high-performance engine developed by Google for Chrome. V8 compiles JavaScript into machine code, so it runs very fast.

Component 2: libuv

libuv is a C library that gives Node.js its asynchronous capabilities. It handles the event loop, file system operations, networking, and a background **thread pool** for heavy tasks.

Component 3: Single-Threaded Event Loop

Node.js runs your JavaScript on a **single main thread**. Instead of waiting for slow operations (reading a file, calling an API, querying a database), it hands them off and keeps working. When they finish, their callbacks are queued and executed.

The Execution Flow

1. Your code enters the **Call Stack** and runs line by line.
2. When a slow task appears (such as an API call), Node.js passes it to the background system (libuv / OS).
3. The main thread **does not wait**. It continues with the next line of code.
4. When the background task completes, its callback goes into a **queue**.
5. The **Event Loop** continuously checks whether the call stack is empty. When it is, the loop moves the queued callback onto the stack to run.

Demonstration

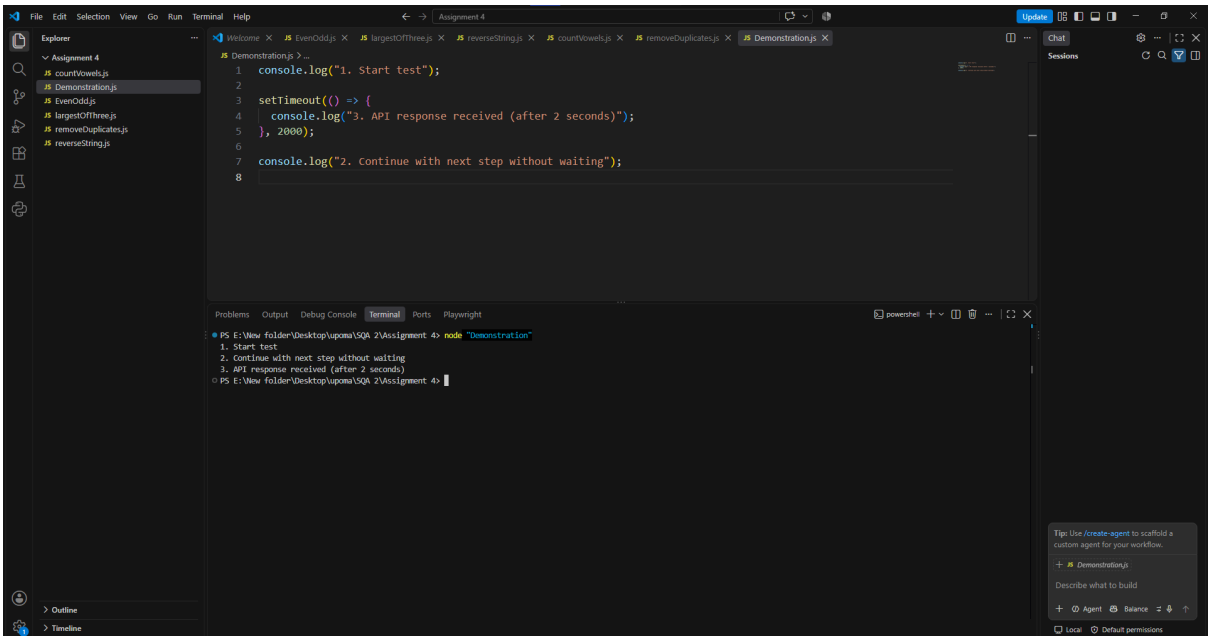
```
console.log("1. Start test");

setTimeout(() => {
```

```
  console.log("3. API response received (after 2 seconds)");
}, 2000);

console.log("2. Continue with next step without waiting");
```

Output:



Line 3 prints last even though it appears in the middle of the code. This is **non-blocking, asynchronous behavior**.

Key Characteristics

Concept	Meaning
Non-blocking I/O	Slow operations don't freeze the program
Event-driven	Code reacts to events (a response arriving, a file finishing reading)
Single-threaded	One main thread runs JavaScript, with background help for I/O
Asynchronous	Uses callbacks, Promises, and `async/await`

2.3 Advantages of Node.js for QA Automation

1. **One language across the whole stack.** Most web applications use JavaScript on the frontend. Testers using Node.js can write tests in the same language as the developers, which makes code reviews and collaboration easier.
2. **Rich tool ecosystem (npm).** The **npm** registry gives access to leading test tools: **Playwright, Cypress, Puppeteer, WebdriverIO** for UI/end-to-end testing; **Jest, Mocha, Chai** for unit and assertion testing; **Supertest, Axios, Newman (Postman CLI)** for API testing.
3. **Excellent for API testing.** Node.js handles HTTP requests asynchronously, so many API calls can run efficiently. JSON is JavaScript's native data format, which makes reading and validating API responses simple.

4. **Fast execution and parallelism.** Non-blocking I/O and the V8 engine let test runners execute many tests quickly, and tools like Playwright run tests in parallel to shorten regression cycles.
5. **Easy CI/CD integration.** Node.js is supported by Jenkins, GitHub Actions, GitLab CI, and Azure DevOps. Installing dependencies and running tests takes only `npm install` and `npm test`.
6. **Cross-platform.** The same test suite runs on Windows, macOS, and Linux, which matters when teams use different machines.
7. **File system and data handling.** Node.js can read test data from JSON/CSV/Excel files, write logs, and generate HTML/JSON reports.
8. **Large community and free tooling.** It is open source, with abundant documentation and quick fixes for common problems.
9. **Supports AI-augmented testing.** Node.js SDKs make it straightforward to connect AI services to test workflows, for example generating test data or analyzing failures.

2.4 Difference Between Browser JavaScript and Node.js

Feature	Browser JavaScript	Node.js
Purpose	Makes web pages interactive (client side)	Runs JavaScript on servers, machines, and CI tools (server side/backend)
Environment	Runs inside a browser (Chrome, Firefox, Edge)	Runs in a standalone runtime on the operating system
JavaScript Engine	Depends on browser (V8 in Chrome, SpiderMonkey in Firefox)	V8 engine
Global Object	`window`	`global` (or `globalThis`)
DOM Access	Yes (`document.getElementById()`)	No DOM, since there is no web page
Browser APIs	`alert()`, `localStorage`, `fetch`, `window.location`	Not available by default
File System Access	Blocked for security	Full access through the `fs` module
Module System	ES Modules (`import`)	CommonJS (`require`) and ES Modules
Built-in Modules	Limited	`fs`, `http`, `path`, `os`, `crypto`, and more
Package Management	Typically `<script>` tags or bundlers	Uses **npm** directly
Security Model	Sandboxed to protect users	Full system access, so the developer must be careful
How to Run	Open an HTML page in a browser	Run `node file.js` in the terminal
QA Use	Runs code the tester inspects	Runs the test framework itself

Code Comparison

Browser JavaScript (works only in a browser):

```
document.getElementById("loginBtn").click();
alert("Login clicked");
```

Node.js (works only in Node):

```
const fs = require("fs");
fs.writeFileSync("report.txt", "Test execution completed");
console.log("Report file created");
```

Running the browser code in Node throws `ReferenceError: document is not defined`. Running the Node code in a browser throws `require is not defined`. This shows they are different environments running the same language.

Conclusion

Node.js extends JavaScript beyond the browser. For a QA engineer, it powers the automation tools, API testing libraries, and CI/CD pipelines used in modern testing.